



FILE PATH TRAVERSAL

TRAVERSAL SEQUENCES
STRIPPED NON-RECURSIVELY

ACTIVIDAD 09

File Path Traversal: *Traversal Sequences Stripped non-recursively*

Especificaciones del laboratorio

Laboratorio: File Path Traversal, Traversal Sequences stripped non-recursively

Nivel: Practitioner

Tipo de explotación: File Path Traversal, evasión de saneamiento no recursivo de secuencias de traversal mediante secuencias anidadas

Objetivo del laboratorio

Explotar una vulnerabilidad de path traversal presente en la funcionalidad de visualización de imágenes de producto de la aplicación, con el fin de recuperar el contenido del archivo `/etc/passwd` del servidor. La aplicación implementa un filtro que elimina las secuencias de traversal (`../`) del parámetro `filename`, pero lo hace de forma no recursiva, es decir, aplica la eliminación una sola vez sin volver a evaluar el resultado. El objetivo específico es construir un payload capaz de evadir dicho filtro aprovechando esta debilidad en su lógica de saneamiento.

Marco Teórico

Mecanismo del filtro y su bypass mediante ruta absoluta

Nuevamente a diferencia del caso simple, esta página sí intenta detectar y eliminar la secuencia `../` del valor de `filename` antes de resolver la ruta del archivo. Sin embargo, esta defensa parte del supuesto que la eliminación de la secuencia solo necesita ejecutarse una vez sobre el string recibido.

El propio PortSwigger documenta esta clase de bypass como uno de los obstáculos comunes al explotar path traversal ya que cuando el filtro elimina las secuencias de traversal de forma no recursiva, es posible anidar una secuencia de traversal dentro de otra, de modo que, al eliminarse la secuencia interna, los caracteres restantes se recompongan en una nueva secuencia válida (PortSwigger, s.f.).

La causa técnica suele ser una llamada de saneamiento del tipo "reemplazar `../` por vacío" ejecutada una sola vez sobre el string completo, en lugar de repetirse en bucle hasta que ya no queden coincidencias. Al recibir el payload `....//`, la función localiza la subcadena `../` contenida en su interior (los caracteres 2 a 4) y la elimina, pero no vuelve a examinar el resultado de esa sustitución. Lo que queda tras la eliminación, un punto inicial seguido de la barra final (`".." + "/"`), se recompone en una secuencia `../` perfectamente válida, que el filtro nunca llega a detectar porque su verificación ya terminó. Repitiendo este bloque en cada nivel de la ruta (`....//....//....//`), el servidor termina procesando `../..../` antepuesto a `etc/passwd`, retrocediendo desde el directorio de imágenes hasta la raíz del sistema de archivos.

Clasificación según OWASP Top 10 y triada CIA

Al igual que en el caso simple, esta vulnerabilidad corresponde a la categoría **A01:2025 – Broken Access Control del OWASP Top 10 de 2025**, la categoría de mayor incidencia registrada, presente en el 100% de las aplicaciones evaluadas (OWASP Foundation, 2025), y sigue mapeada a CWE-22 (MITRE Corporation, 2024). La diferencia frente al caso del bypass por ruta absoluta no es la categoría ni el archivo objetivo, sino el tipo de defecto en la defensa, aquí el filtro sí cubre el vector correcto de secuencias relativas, pero lo hace con un algoritmo de una sola pasada que no contempla que su propia eliminación puede generar una nueva secuencia explotable.

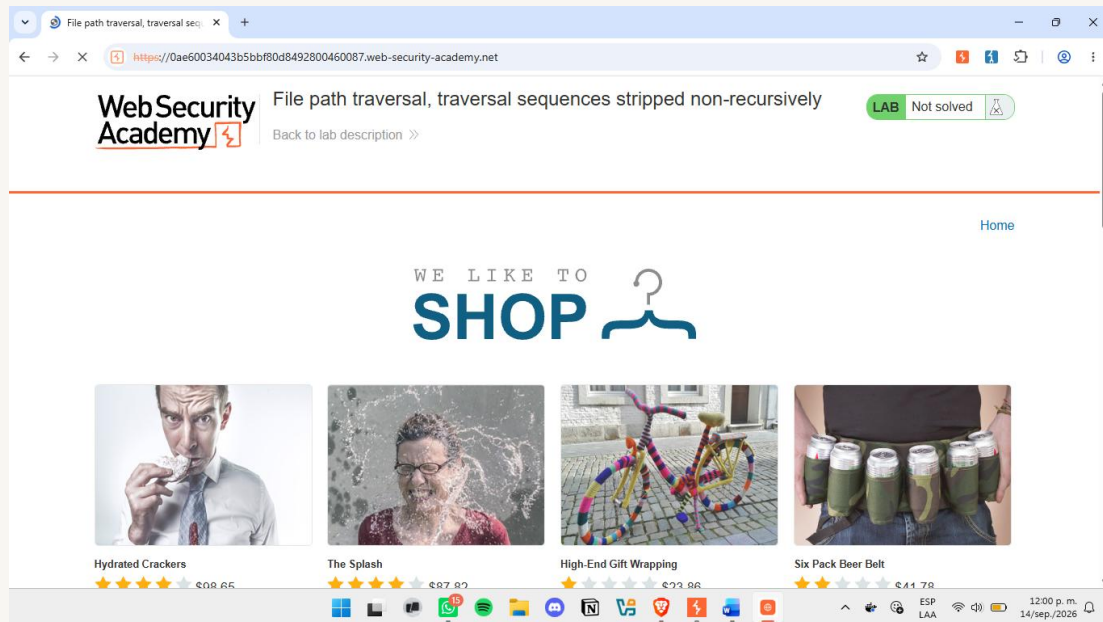
El impacto sobre la triada CIA es el mismo que en los casos previos pues compromete la Confidencialidad al exponer archivos internos del servidor que no deberían ser accesibles desde la capa de presentación de la aplicación. Lo que cambia es el fondo del problema, un control de seguridad que depende de una única operación de limpieza de texto es frágil por diseño, porque su corrección depende del supuesto de que basta con una única saneación, que no siempre es válido frente a un input adversarial construido específicamente para violarlo.

Riesgos en un entorno real

Este patrón, sanear un input eliminando un patrón peligroso con una sola operación de reemplazo, es una práctica frecuente en aplicaciones reales, sobre todo cuando el saneamiento se agrega como parche rápido tras detectar un path traversal en una auditoría, sin rediseñar la validación desde cero. El riesgo práctico es que una prueba superficial (probar ../../etc/passwd directamente) parece confirmar que la vulnerabilidad quedó corregida, ya que el filtro efectivamente la bloquea, mientras que la misma exposición del archivo /etc/passwd, /etc/shadow o de archivos de configuración internos sigue siendo alcanzable con un payload ligeramente más elaborado que cualquier atacante con conocimiento de esta clase de bypass puede construir.

Procedimiento

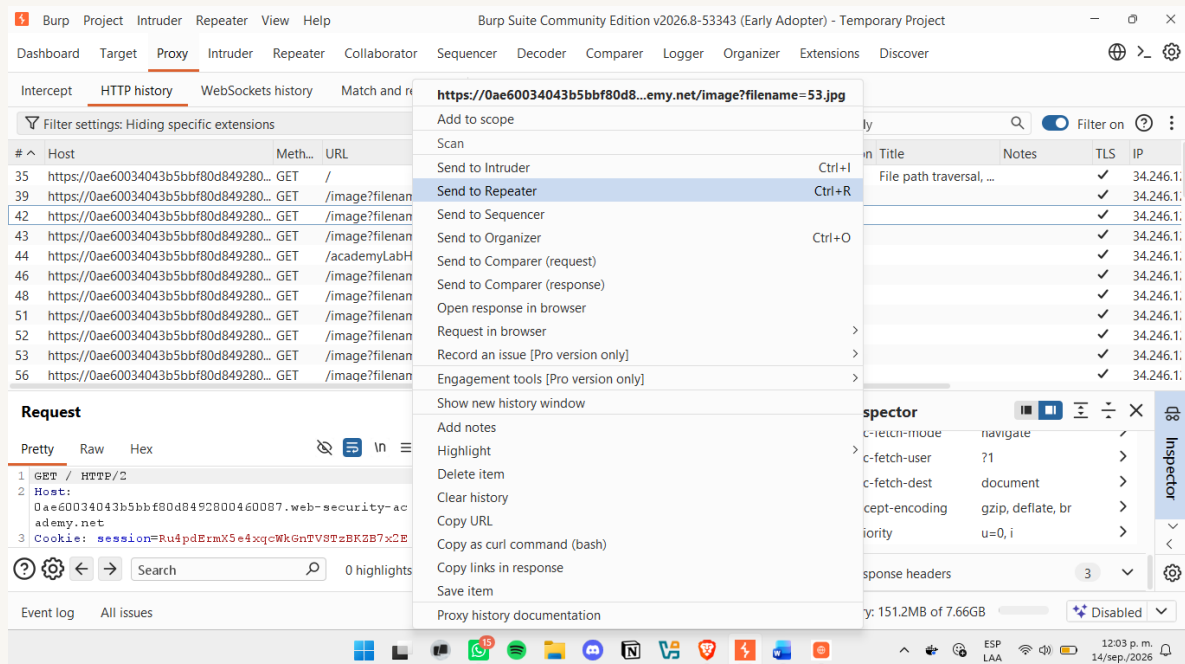
1. Se accede a la instancia del laboratorio "File path traversal, traversal sequences stripped non-recursively" de PortSwigger Web Security Academy. El laboratorio inicia en estado "Not solved" y presenta una tienda en línea con productos que incluyen imágenes cargadas dinámicamente.



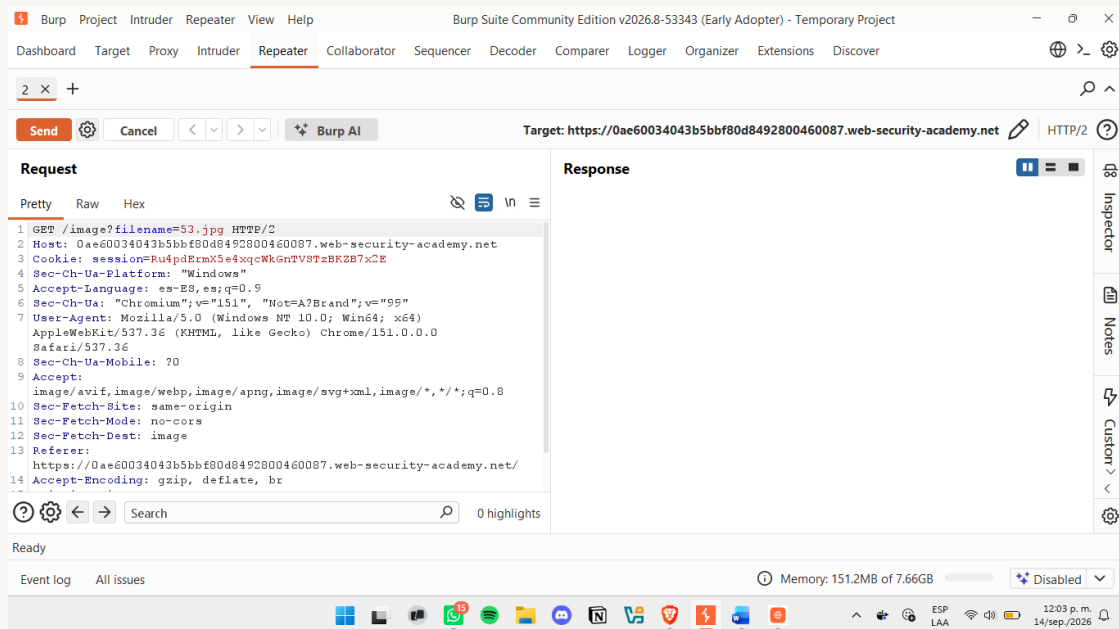
2. Con Burp Suite en modo de interceptación de tráfico, se navega por la tienda y se revisa el historial de proxy. Se identifica que cada imagen de producto se carga mediante una solicitud GET al endpoint `/image`, pasando el nombre del archivo en el parámetro `filename` (por ejemplo, `/image?filename=53.jpg`).

35	https://0ae60034043b5bbf80d849280...	GET	/		200	10616	HTML
39	https://0ae60034043b5bbf80d849280...	GET	/image?filename=52.jpg	✓	200	143243	JPEG
42	https://0ae60034043b5bbf80d849280...	GET	/image?filename=53.jpg	✓	200	301401	JPEG
43	https://0ae60034043b5bbf80d849280...	GET	/image?filename=26.jpg	✓	200	220040	JPEG
44	https://0ae60034043b5bbf80d849280...	GET	/academyLabHeader		101	147	
46	https://0ae60034043b5bbf80d849280...	GET	/image?filename=38.jpg	✓	200	277107	JPEG
48	https://0ae60034043b5bbf80d849280...	GET	/image?filename=51.jpg	✓	200	177099	JPEG
51	https://0ae60034043b5bbf80d849280...	GET	/image?filename=46.jpg	✓	200	274394	JPEG
52	https://0ae60034043b5bbf80d849280...	GET	/image?filename=23.jpg	✓	200	266000	JPEG
53	https://0ae60034043b5bbf80d849280...	GET	/image?filename=31.jpg	✓	200	127667	JPEG
56	https://0ae60034043b5bbf80d849280...	GET	/image?filename=30.jpg	✓	200	446412	JPEG

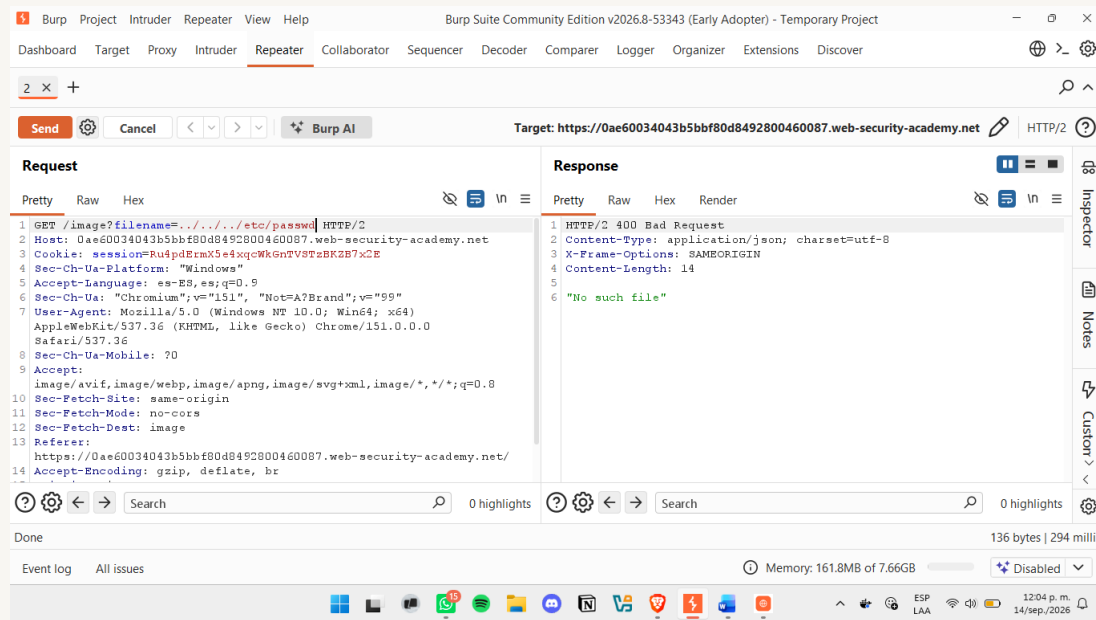
3. Se selecciona una de las solicitudes de imagen en el historial y, mediante el menú contextual, se envía al módulo Repeater para poder modificar y reenviar la solicitud de forma controlada.



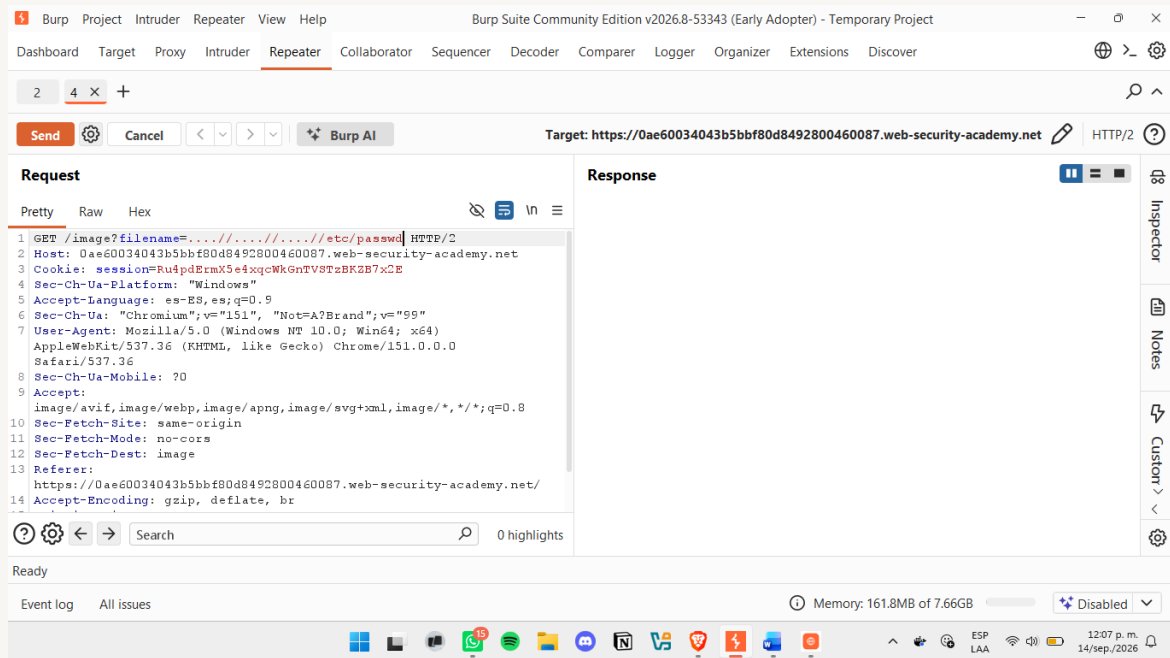
4. En Repeater se visualiza la solicitud original sin modificar: GET /image?filename=53.jpg, junto con las cabeceras enviadas por el navegador, incluyendo la cookie de sesión del laboratorio.



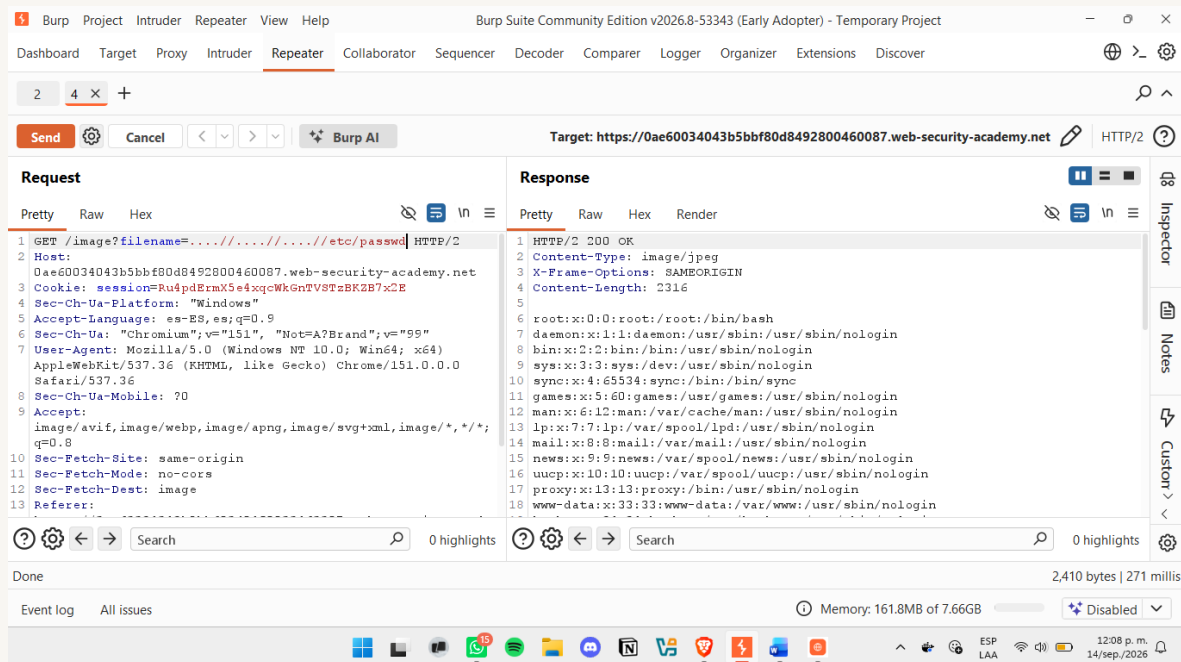
5. Se modifica el parámetro filename con la secuencia de traversal estándar ../../etc/passwd y se envía la solicitud. El servidor responde con un código 400 Bad Request y el mensaje "No such file", lo que confirma que la aplicación cuenta con un filtro que bloquea o elimina las secuencias de traversal directas.



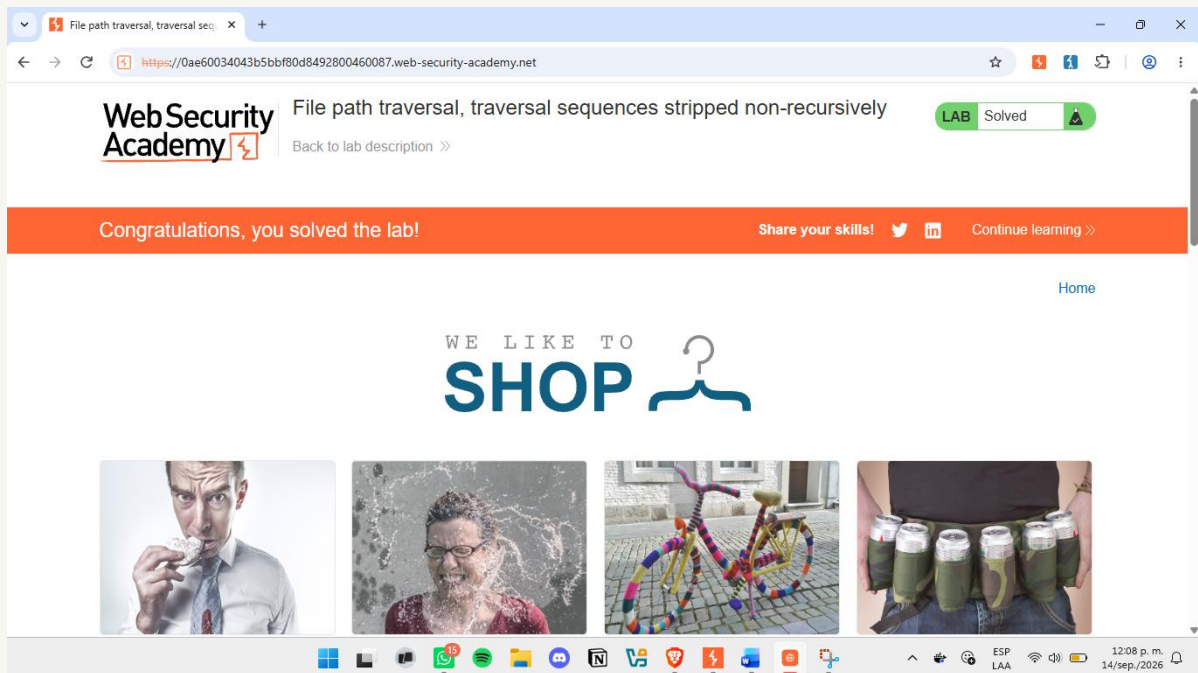
6. Dado que el filtro elimina las secuencias `../` de forma no recursiva, se construye un payload con secuencias anidadas: `....//....//....//etc/passwd`. Al eliminarse el `../` interno de cada bloque `....`, el resultado se compone en una secuencia `../` válida, evadiendo así el saneamiento.



7. Al enviar la solicitud con el payload de bypass, el servidor responde con un código 200 OK y devuelve en el cuerpo de la respuesta el contenido completo del archivo `/etc/passwd` del sistema, incluyendo las entradas de los usuarios del sistema operativo (`root`, `daemon`, `bin`, `sys`, entre otros).



8. Al regresar a la aplicación, el laboratorio muestra el estado "Solved" junto con el mensaje "Congratulations, you solved the lab!", confirmando que la explotación fue exitosa.



Explicación del Payload

El payload utilizado fue `../../../../../../../../etc/passwd`, colocado como valor completo del parámetro `filename`. A diferencia del bypass por ruta absoluta, aquí sí están presentes secuencias de traversal, solo que no aparecen como el patrón literal `"../"` que el filtro busca, sino envueltas dentro de un bloque más largo (`../../../../`) que las oculta de una revisión hecha en una sola pasada.

Su efectividad depende de que el saneamiento se implemente como una operación de búsqueda y reemplazo ejecutada una única vez sobre el string recibido, en lugar de repetirse hasta que ya no queden coincidencias. Es un comportamiento común en implementaciones que asumen que basta con "limpiar" el input una sola vez entonces al quitar la subcadena ../ del interior de cada bloque, los caracteres que quedan a los lados se juntan y forman de nuevo una secuencia ../ completa, la cual ya no vuelve a pasar por el filtro.

El filtro contra "../" sí llega a activarse con este payload, a diferencia del caso de ruta absoluta, y de hecho elimina exactamente lo que fue diseñado para eliminar. El problema no está en que ignore el patrón, sino en que su lógica de una sola pasada no contempla que el propio acto de eliminar una coincidencia puede generar una coincidencia nueva, dejando desprotegido el vector de secuencias anidadas.

Mitigación recomendada

Las mismas medidas de fondo aplicables al caso simple siguen siendo válidas aquí, con un énfasis adicional en el motivo por el cual este filtro específico resultó insuficiente:

- **Evitar el paso directo de entradas a APIs del sistema de archivos:** en lugar de dejar que el cliente mande cualquier ruta o nombre de archivo, usar un identificador aleatorio (UUID) que el servidor traduce internamente contra una base de datos para encontrar el archivo real.
- **Validación estricta por lista blanca, no por bloqueo de patrones:** en lugar de intentar limpiar el input quitando secuencias específicas como "../" (un enfoque de blacklist que este laboratorio demuestra que es frágil incluso cuando sí contempla el vector correcto), aceptar únicamente caracteres alfanuméricos y una extensión de archivo predefinida, rechazando cualquier valor que contenga "/", "\", "." repetidos o que no cumpla ese formato exacto.
- **Saneamiento en bucle hasta punto fijo, si el reemplazo de patrones es inevitable:** cuando por alguna razón se deba seguir eliminando secuencias como "../" del input, la eliminación debe repetirse en un ciclo hasta que una nueva pasada ya no produzca ningún cambio en el string, en lugar de ejecutarse una sola vez; esto cierra específicamente la clase de bypass basada en secuencias anidadas explotada en este laboratorio.
- **Canonicalización y verificación de ruta base:** resolver la ruta absoluta canónica resultante mediante funciones nativas del lenguaje de desarrollo, como por ejemplo getCanonicalPath() en Java o os.path.realpath() / Path.resolve() en Python, y verificar que dicho resultado comience estrictamente dentro del directorio base autorizado, sin depender de que el saneamiento previo del string haya sido correcto.
- **Principio de menor privilegio:** ejecutar el proceso del servidor con permisos mínimos, de forma que, aunque una secuencia de traversal lograra evadir la validación de la aplicación, el sistema operativo continúe negando el acceso a directorios sensibles como /etc o /root.
- **Auditoría y pruebas específicas de bypass:** al probar defensas anti-traversal, incluir explícitamente casos de secuencias anidadas, codificaciones alternas y rutas absolutas dentro de las pruebas de penetración, en lugar de validar la corrección del filtro únicamente contra el vector de ataque original que motivó su implementación.

Referencias

MITRE Corporation. (2024). *CWE-22: Improper limitation of a pathname to a restricted directory ('Path Traversal')*. Common Weakness Enumeration. <https://cwe.mitre.org/data/definitions/22.html>

OWASP Foundation. (2025). *A01:2025 – Broken Access Control*. OWASP Top 10:2025. https://owasp.org/Top10/2025/A01_2025-Broken_Access_Control/

PortSwigger. (s. f.). *Lab: File path traversal, traversal sequences stripped non-recursively*. Web Security Academy. <https://portswigger.net/web-security/file-path-traversal/lab-sequences-stripped-non-recursively>

PortSwigger. (s.f.). *What is path traversal, and how to prevent it?* Web Security Academy. <https://portswigger.net/web-security/file-path-traversal>